

Python Complete Notes (Basics → OOP)

1. Introduction to Python

Python is a **high-level, interpreted programming language** created by Guido van Rossum in **1991**.

Features

- Easy to learn
- Simple syntax
- Platform independent
- Large libraries
- Used in AI, Data Science, Web, Automation

Example Program

```
print("Hello World")
```

Output

Hello World

2. Variables

Variables store data.

Syntax

```
variable_name = value
```

Example

```
name = "Rupa"  
age = 23  
salary = 25000
```

Rules

- Cannot start with number

- No spaces
- Case sensitive

Valid

name
user_age
student1

Invalid

1name
user age

3. Data Types

1. Integer

Whole numbers

x = 10

2. Float

Decimal numbers

y = 10.5

3. String

Text data

name = "Python"

4. Boolean

is_active = True

Check Data Type

print(type(x))

4. Type Casting

Convert one data type to another.

Example

```
x = "10"
```

```
y = int(x)
```

```
print(y)
```

Types

```
int()
```

```
float()
```

```
str()
```

Example

```
a = 5
```

```
b = float(a)
```

```
print(b)
```

Output

```
5.0
```

5. Input Function

Used to take input from user.

```
name = input("Enter your name: ")
```

```
print(name)
```

Number input

```
age = int(input("Enter age: "))
```

6. Operators

1. Arithmetic Operators

Operator	Example
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Power

Example

```
a = 10
b = 5

print(a + b)
print(a * b)
```

2. Comparison Operators

```
==
!=
>
<
>=
<=
```

Example

```
a = 10
b = 5

print(a > b)
```

3. Logical Operators

and
or
not

Example

```
x = 10
```

```
print(x > 5 and x < 20)
```

7. Conditional Statements

Used for decision making.

if

```
age = 18
```

```
if age >= 18:  
    print("Eligible to vote")
```

if else

```
age = 16
```

```
if age >= 18:  
    print("Eligible")  
else:  
    print("Not eligible")
```

if elif else

```
marks = 80
```

```
if marks >= 90:  
    print("Grade A")  
elif marks >= 70:  
    print("Grade B")  
else:  
    print("Grade C")
```

8. Loops

Loops repeat code.

For Loop

```
for i in range(5):  
    print(i)
```

Output

```
0  
1  
2  
3  
4
```

Example

```
for i in range(1,6):  
    print(i)
```

While Loop

```
i = 1
```

```
while i <= 5:  
    print(i)  
    i += 1
```

9. Break and Continue

Break

Stops loop.

```
for i in range(10):
```

```
    if i == 5:
        break
```

```
    print(i)
```

Continue

Skip iteration.

```
for i in range(5):
```

```
    if i == 2:
        continue
```

```
    print(i)
```

10. Functions

Function is reusable code block.

Syntax

```
def function_name():
    code
```

Example

```
def greet():
    print("Hello")
```

```
greet()
```

Function with Parameters

```
def greet(name):
    print("Hello", name)
```

```
greet("Rupa")
```

Function with Return

```
def add(a,b):  
    return a + b  
  
result = add(10,5)  
  
print(result)
```

11. Types of Functions

1. Built-in Function

Examples

```
print()  
len()  
type()
```

2. User Defined Function

Functions created by user.

```
def square(n):  
    return n*n
```

3. Lambda Function

Small anonymous function.

```
square = lambda x: x*x  
  
print(square(5))
```


Output

25

12. Types of Arguments

1. Positional Argument

```
def add(a,b):  
    print(a+b)
```

```
add(5,3)
```

2. Keyword Argument

```
add(a=5,b=3)
```

3. Default Argument

```
def greet(name="Guest"):  
    print(name)
```

```
greet()
```

4. Variable Arguments

***args**

```
def add(*numbers):  
    print(sum(numbers))
```

```
add(1,2,3,4)
```

****kwargs**

```
def details(**data):
```

```
    print(data)
```

```
details(name="Rupa", age=23)
```

13. Lists

List stores multiple values.

Example

```
numbers = [10,20,30]
```

Access

```
print(numbers[0])
```

List Methods

append()

```
numbers.append(40)
```

insert()

```
numbers.insert(1,15)
```

remove()

```
numbers.remove(20)
```

pop()

```
numbers.pop()
```

sort()

```
numbers.sort()
```

reverse()

numbers.reverse()

14. Tuple

Tuple is **immutable list**.

t = (10,20,30)

Cannot modify.

Example

print(t[1])

15. Set

Set stores **unique values**.

s = {1,2,3,4}

Example

s.add(5)

Set removes duplicates automatically.

{1,1,2,3} → {1,2,3}

16. Dictionary

Stores **key value pairs**.

Example

```
student = {  
    "name": "Rupa",  
    "age": 23  
}
```

Access value

```
print(student["name"])
```

Add value

```
student["city"] = "Hyderabad"
```

17. File Handling

Create / Write File

```
file = open("data.txt", "w")
```

```
file.write("Hello")
```

```
file.close()
```

Read File

```
file = open("data.txt", "r")
```

```
print(file.read())
```

```
file.close()
```

Best Method (with)

with open("data.txt", "r") as file:

```
    print(file.read())
```

18. Exception Handling

Handles errors.

Example

try:

```
num = int(input("Enter number"))
```

```
result = 10 / num
```

```
print(result)
```

except ZeroDivisionError:

```
print("Cannot divide by zero")
```

except ValueError:

```
print("Invalid input")
```

19. Object Oriented Programming (OOP)

OOP is programming using **objects** and **classes**.

Class and Object

Class

Blueprint of object.

Object

Instance of class.

Example

```
class Student:
```

```
name = "Rupa"

s1 = Student()

print(s1.name)
```

Constructor

Constructor runs automatically.

```
class Student:

    def __init__(self,name):

        self.name = name

s1 = Student("Rupa")

print(s1.name)
```

Methods

```
class Student:

    def greet(self):

        print("Hello Student")

s1 = Student()

s1.greet()
```

Inheritance

Child class inherits parent class.

```
class Animal:
```

```
def sound(self):
    print("Animal sound")

class Dog(Animal):

    def bark(self):
        print("Dog barks")

d = Dog()

d.sound()
d.bark()
```

Polymorphism

Same function different behavior.

```
class Dog:

    def sound(self):
        print("Bark")

class Cat:

    def sound(self):
        print("Meow")

for animal in [Dog(),Cat()]:

    animal.sound()
```

Encapsulation

Hiding internal data.

```
class Bank:

    def __init__(self):

        self.__balance = 1000
```

```
def show_balance(self):  
  
    print(self.__balance)  
  
b = Bank()  
  
b.show_balance()
```

Abstraction

Showing only essential details.

Example using abstract class.

```
from abc import ABC, abstractmethod  
  
class Shape(ABC):  
  
    @abstractmethod  
    def area(self):  
        pass
```

Summary

Topics covered

- Python Introduction
- Variables
- Data Types
- Operators
- Conditions
- Loops
- Functions
- Arguments
- Lists
- Tuple
- Set
- Dictionary
- File Handling
- Exception Handling

- OOP Concepts
 - Class
 - Object
 - Constructor
 - Inheritance
 - Polymorphism
 - Encapsulation
 - Abstraction

1.

RupaLife